UFPI/CSHNB - Redes II
Segunda Avaliação

Especificação do Trabalho Prático: Estudo Experimental de Desempenho de Protocolos Web (HTTP/1.1 vs HTTPS/TLS vs HTTP/3-QUIC) em Ambiente Emulado com Docker e Python

1. Visão Geral e Objetivos Pedagógicos

Este Trabalho Prático — que corresponde a 50% da pontuação total da 2ª Avaliação (N2) — foi elaborado para conduzir os estudantes da abstração conceitual à análise experimental rigorosa dos protocolos das camadas de aplicação e de transporte que estruturam a Web moderna. A avaliação do desempenho sob condições de rede rigorosamente controladas permite compreender como as decisões de arquitetura de software e de projeto de protocolos afetam a entrega de dados em escala global, conectando a teoria às métricas empíricas de infraestrutura.

O objetivo central do projeto é desenvolver competências avançadas em engenharia de desempenho e de diagnóstico de redes. Com base no referencial teórico do livro *Redes de Computadores (Livro Texto)*, os alunos deverão investigar e mensurar quantitativamente três fenômenos fundamentais:

- **Overhead de Handshakes e Latência Inicial:** O estabelecimento de uma conexão TCP convencional exige a troca de mensagens de controle em três vias (*3-way handshake*), consumindo exatamente **1 RTT** (*Round-Trip Time*) antes do envio do primeiro byte de dados da aplicação. Quando a camada de segurança TLS 1.3 é acoplada sobre o TCP (como no HTTPS/HTTP/2), adiciona-se obrigatoriamente **1 RTT adicional** para a negociação de chaves criptográficas e de certificados, totalizando **2 RTTs cumulativos** de latência inicial para a conclusão do handshake combinado. Em contrapartida, o HTTP/3 funde as camadas de transporte e de segurança no protocolo QUIC (que opera sobre datagramas UDP), realizando a negociação conjunta em um único procedimento integrado de **1 RTT** (ou **0-RTT** em cenários de reconexão via *session resumption*). Os alunos avaliarão o impacto direto dessa redução de RTTs no *Time to First Byte* (TTFB) e na Qualidade de Experiência (*Quality of Experience* - QoE) percebida pelo usuário final.
- **Bloqueio de Início de Linha (*Head-of-Line - HoL Blocking*):** No HTTP/1.1, o HoL blocking manifesta-se prioritariamente na **Camada de Aplicação**, onde as requisições enviadas sobre o mesmo canal TCP precisam ser processadas e respondidas em ordem estritamente sequencial (ou mitigadas de forma ineficiente por meio da abertura de múltiplas conexões TCP paralelas). O HTTP/2 resolveu o bloqueio na camada de aplicação ao introduzir a multiplexação por quadros sobre uma única conexão TCP; contudo, deslocou a vulnerabilidade para a **Camada de Transporte**: como o TCP garante

a entrega ordenada do buffer, a perda de um único segmento TCP força o sistema operacional a reter todos os pacotes subsequentes de todos os fluxos multiplexados até que a retransmissão do pacote perdido seja concluída. O HTTP/3-QUIC elimina o HoL blocking em ambas as camadas ao implementar fluxos (*streams*) UDP totalmente independentes e isolados, em que a perda de um datagrama afeta exclusivamente o fluxo ao qual pertence.

- **Efeito das Condições do Canal:** A variação do RTT e a injeção controlada de perda de pacotes expõem os limites operacionais das abordagens orientadas à conexão baseadas em TCP, em comparação com a resiliência do QUIC sobre UDP, especialmente no que diz respeito aos algoritmos de controle de congestionamento e aos mecanismos de recuperação de perdas.

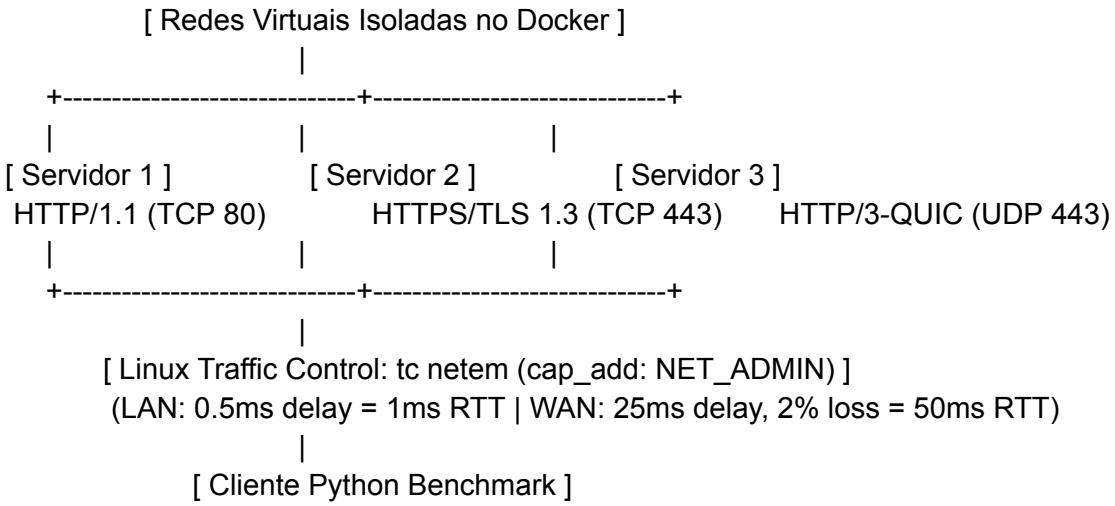
A tabela comparativa a seguir sintetiza as diferenças arquiteturais que deverão ser validadas e quantificadas ao longo dos experimentos:

Protocolo Web	Protocolo de Transporte Base	Mecanismo de Segurança	Mecanismo de Multiplexação	Suscetibilidade ao HoL Blocking
HTTP/1.1	TCP	Opcional (TLS sobre TCP)	Inexistente (requer conexões TCP paralelas)	**Alta** (Restrita à Camada de Aplicação)
HTTPS (HTTP/2)	TCP	Obrigatório (TLS 1.3)	Multiplexação por quadros em conexão única	**Média/Alta** (Transferida para a Camada de Transporte TCP)
HTTP/3	UDP (QUIC)	Integrado (TLS 1.3 sobre QUIC)	Fluxos (*streams*) nativos e independentes	**Nula** (Eliminado nas camadas de Aplicação e Transporte)

Munidos dessa fundamentação conceitual, a próxima seção detalha a arquitetura do ambiente virtualizado necessário para isolar os experimentos de interferências externas e garantir a reprodutibilidade científica dos testes.

2. Arquitetura do Ambiente e Infraestrutura Docker

A obtenção de medições com validade científica exige um ambiente experimental rigorosamente isolado e reprodutível. Para evitar que ruídos de rede local, oscilações do provedor de Internet ou comportamentos de cache do sistema operacional distorçam os dados, toda a infraestrutura será emulada por meio de contêineres Docker isolados, orquestrados via `docker-compose`.



Especificações de Orquestração (`docker-compose.yml`) e Permissões de Kernel

O arquivo `docker-compose.yml` deve estruturar três contêineres de servidor Web distintos, alocados em redes virtuais Docker dedicadas. **Atenção de Engenharia:** Para que as ferramentas de manipulação de tráfego do kernel Linux funcionem no interior dos contêineres sem falhas de permissão (`RTNETLINK answers: Operation not permitted`), é **obrigatório** declarar a funcionalidade `cap_add: - NET_ADMIN` na especificação de cada serviço.

Abaixo apresenta-se o trecho de referência para a definição da infraestrutura:

```
version: '3.8'

services:
  server-http1:
    image: nginx:alpine
    container_name: server_http11
    cap_add:
      - NET_ADMIN
    ports:
      - "8080:80"
    networks:
      net_bench:
        ipv4_address: 172.20.0.10

  server-http2:
    image: nginx:alpine
    container_name: server_http2_tls
    cap_add:
      - NET_ADMIN
    ports:
      - "8443:443"
    volumes:
      - ./certs:/etc/nginx/certs
```

```
    networks:
      net_bench:
        ipv4_address: 172.20.0.20

  server-http3:
    image: caddy:2.7-alpine
    container_name: server_http3_quic
    cap_add:
      - NET_ADMIN
    ports:
      - "9443:443/tcp"
      - "9443:443/udp"
    volumes:
      - ./certs:/etc/caddy/certs
    networks:
      net_bench:
        ipv4_address: 172.20.0.30

networks:
  net_bench:
    driver: bridge
    ipam:
      config:
        - subnet: 172.20.0.0/16
```

Emulação de Rede com Linux Traffic Control (`tc netem`)

A alteração do comportamento do canal de comunicação será realizada injetando regras de enfileiramento diretamente na interface de rede virtual (`eth0`) dos contêineres utilizando o utilitário `tc` com o módulo `netem` (*Network Emulator*).

Nota Explicativa sobre Cálculo de RTT: O comando `tc netem delay Xms` aplica o atraso bidirecionalmente em sentido unidirecional por padrão na saída da interface. Portanto, para emular um RTT de N ms, o atraso injetado na interface deve ser exatamente N/2 ms.

- **Cenário 1 (Rede Local - LAN):** Emulação de um canal ideal de alta velocidade. Atraso unidirecional de `0.5ms` (resultando em **RTT total de 1ms**) e **0% de perda de pacotes**.
- **Cenário 2 (Rede Longa/Móvel - WAN):** Emulação de canal sujeito a latência e degradação. Atraso unidirecional de `25ms` (resultando em **RTT total de 50ms**) e **2% de taxa de perda sintética de pacotes**.

Comandos Sintáticos Linux para Aplicação e Limpeza de Regras

Os comandos executados nos scripts de automação do ambiente devem seguir a sintaxe exata descrita abaixo:

- **Aplicação do Cenário 1 (LAN - Latência RTT de 1ms, 0% Perda):**

```
# Aplica atraso unidirecional de 0.5ms (RTT final = 1.0ms)
docker exec --privileged server_http11 tc qdisc add dev eth0 root netem delay 0.5ms
docker exec --privileged server_http2_tls tc qdisc add dev eth0 root netem delay 0.5ms
docker exec --privileged server_http3_quic tc qdisc add dev eth0 root netem delay 0.5ms
```

- **Aplicação do Cenário 2 (WAN - Latência RTT de 50ms, 2% de Perda):**

```
# Aplica atraso unidirecional de 25ms (RTT final = 50ms) e 2% de perda estocástica
docker exec --privileged server_http11 tc qdisc add dev eth0 root netem delay 25ms loss 2%
docker exec --privileged server_http2_tls tc qdisc add dev eth0 root netem delay 25ms loss 2%
docker exec --privileged server_http3_quic tc qdisc add dev eth0 root netem delay 25ms loss 2%
```

- **Limpeza e Restauração da Interface (Estado Limpo):**

```
# Delita a disciplina de enfileiramento (qdisc) e restaura o tráfego normal
docker exec --privileged server_http11 tc qdisc del dev eth0 root
docker exec --privileged server_http2_tls tc qdisc del dev eth0 root
docker exec --privileged server_http3_quic tc qdisc del dev eth0 root
```

Impacto Esperado no Transporte e Congestionamento

No Cenário 2 (WAN), a injeção de uma taxa de perda de 2% causará o colapso da vazão útil do TCP. Como o algoritmo TCP CUBIC interpreta qualquer perda de pacote como um sinal inequívoco de congestionamento na rede, ele reduz drasticamente sua janela de congestionamento (*cwnd*). O estol de retransmissão exigido pela entrega em ordem do TCP congelará os buffers da aplicação. Espera-se que o HTTP/3 sobre QUIC demonstre throughput e RPS substancialmente superiores sob perda de 2%, pois o QUIC desacopla o controle de congestionamento do fluxo individual e utiliza estimativas contínuas baseadas na taxa/largura de banda e no RTT mínimo, permitindo a progressão contínua dos fluxos não afetados.

Estabelecidas a infraestrutura de servidores e as condições de controle do canal, a seção a seguir especifica os requisitos do cliente automatizado em Python.

3. Cliente de Benchmark e Automação de Testes em Python

Navegadores comerciais são inadequados para bancadas de teste de redes devido a interferências de renderização de código, caches locais agressivos e otimizações heurísticas imprevisíveis. Assim, os alunos devem desenvolver uma ferramenta de benchmark dedicada em Python que garanta controle total sobre o ciclo de vida das conexões e sobre a amostragem de dados.

Especificações Técnicas e Módulos Python

A aplicação cliente deve ser construída utilizando chamadas assíncronas e bibliotecas especializadas no suporte aos protocolos investigados:

- `httpx` para interações HTTP/1.1 e HTTP/2 (HTTPS).
- `aioquic` e `asyncio` para gerenciamento de eventos assíncronos e controle nativo de fluxos QUIC / HTTP/3.

Diretrizes de Controle de Vício de Sessão (*Client-Side Bias Control*)

Para garantir a acurácia das medições de handshake e multiplexação, o cliente Python **deve**:

1. **Desativar rigorosamente qualquer mecanismo de cache local de HTTP.**
2. **Isolar o Teste de Handshake (Cold TTFB):** Para medir a latência inicial real de conexão (handshake de 1 RTT do TCP, 2 RTTs do TCP+TLS 1.3 e 1 RTT do QUIC), a aplicação deve encerrar e fechar explicitamente a sessão do cliente (`client.close()`) antes de cada requisição individual, impedindo o reuso da conexão TCP/QUIC (*connection pooling*).
3. **Isolar o Teste de Throughput e Multiplexação (Warm Connection):** Para os testes do perfil de carga concorrente, a aplicação deve estabelecer e manter explicitamente uma única conexão persistente (*warm connection*), permitindo avaliar a capacidade de multiplexação de fluxos no HTTP/2 e no HTTP/3, sem o custo de sucessivos handshakes.

Exportação Obrigatória de Dados Brutos

O cliente Python deve, obrigatoriamente, estruturar e exportar as métricas brutas coletadas ao final de cada bateria de testes para arquivos padronizados nos formatos **`results.json`** ou **`results.csv`**. O arquivo gerado conterá os registros com timestamp de cada requisição individual para posterior processamento estatístico.

Mapeamento de Métricas Operacionais

O cliente deve registrar as seguintes métricas com precisão de microssegundos:

- **Time to First Byte (TTFB):** Tempo decorrido entre a emissão do primeiro byte do cabeçalho da resposta e a recepção do primeiro byte do corpo da resposta. Mede o impacto direto do custo dos handshakes.
- **Tempo Total de Carregamento (*Total Load Time*):** Duração decorrida até a recepção completa do último byte do objeto solicitado.

- **Requisições por Segundo (RPS):** Razão entre o número total de requisições concluídas com sucesso e a duração total, em segundos, da janela de teste.
- **Percentis de Latência (p50 e p99):** O percentil p50 representa a mediana do tempo de resposta. O p99 identifica o limite de latência para 99% das requisições, isolando discrepâncias (*outliers*) causadas por retransmissões e perdas de pacotes.

Perfis de Carga de Teste

O cliente automatizado executará obrigatoriamente dois perfis de carga sintética contra cada um dos 3 servidores sob os 2 cenários de rede:

1. **Carga A (Gargalo de Banda / Regime Permanente):** Download contínuo de 1 objeto grande de 50 MB. Avalia o *throughput* sustentado e a eficiência do controle de congestionamento.
2. **Carga B (Gargalo de Latência e Multiplexação):** Download concorrente de 100 objetos pequenos, cada um de 10 KB. Avalia a capacidade de mitigação do HoL blocking, do reuso de canal e do overhead de concorrência.

Matriz Experimental Obrigatória (12 Testes)

A matriz a seguir define as 12 combinações de teste que devem ser executadas e registradas sequencialmente pelo ambiente de automação:

ID	Perfil de Carga	Protocolo Alvo	Cenário de Rede	Foco Analítico Principal
T01	Carga A (50 MB)	HTTP/1.1	Cenário 1 (LAN - 1ms, 0%)	Vazão máxima TCP em canal ideal
T02	Carga A (50 MB)	HTTPS / TLS 1.3	Cenário 1 (LAN - 1ms, 0%)	Custo de cifragem simétrica do TLS 1.3
T03	Carga A (50 MB)	HTTP/3 (QUIC)	Cenário 1 (LAN - 1ms, 0%)	Eficiência do protocolo QUIC sem perdas

T04	Carga A (50 MB)	HTTP/1.1	Cenário 2 (WAN - 50ms, 2%)	Sensibilidade do TCP CUBIC a perdas e latência
T05	Carga A (50 MB)	HTTPS / TLS 1.3	Cenário 2 (WAN - 50ms, 2%)	Degradação do TCP CUBIC + TLS sob perdas
T06	Carga A (50 MB)	HTTP/3 (QUIC)	Cenário 2 (WAN - 50ms, 2%)	Controle de congestionamento do QUIC sob perdas
T07	Carga B (100x10KB)	HTTP/1.1	Cenário 1 (LAN - 1ms, 0%)	Overhead de múltiplos handshakes TCP paralelos
T08	Carga B (100x10KB)	HTTPS / TLS 1.3	Cenário 1 (LAN - 1ms, 0%)	Eficiência da multiplexação HTTP/2 em baixa latência
T09	Carga B (100x10KB)	HTTP/3 (QUIC)	Cenário 1 (LAN - 1ms, 0%)	Desempenho de fluxos QUIC em rede ideal
T10	Carga B (100x10KB)	HTTP/1.1	Cenário 2 (WAN - 50ms, 2%)	Severidade do HoL blocking na Camada de Aplicação
T11	Carga B (100x10KB)	HTTPS / TLS 1.3	Cenário 2 (WAN - 50ms, 2%)	Severidade do HoL blocking na Camada de Transporte TCP
T12	Carga B (100x10KB)	HTTP/3 (QUIC)	Cenário 2 (WAN - 50ms, 2%)	Mitigação total do HoL blocking via fluxos UDP

Concluída a matriz experimental, os resultados processados fundamentarão a estruturação dos entregáveis descritos a seguir.

4. Entregáveis do Projeto e Padrões de Submissão

O trabalho final consiste em dois artefatos obrigatórios: a infraestrutura e os scripts de código-fonte, e o Relatório Técnico conclusivo e o vídeo de 15 minutos explicando a execução, os pontos importantes da implementação, os resultados e a conclusão dos experimentos.

Entregável 1: Repositório de Código e Infraestrutura (Zip / Git)

O código entregue deve conter a seguinte estrutura de arquivos:

- Arquivo `docker-compose.yml` com as definições de rede, portas e a declaração `cap_add: - NET_ADMIN`.
- Scripts Shell parametrizados para aplicar e limpar as regras do `tc netem`.
- Código-fonte completo do cliente de benchmark em Python com suporte a exportação para `results.json` ou `results.csv`.
- **Script de Automação Runner de Comando Único (`run_experiments.sh`):** Um script mestre que deve, com um único comando: (1) Subir os contêineres Docker; (2) Iterar sobre os cenários do `tc netem`; (3) Executar os 12 testes da matriz experimental através do cliente Python; (4) Salvar os dados brutos exportados; e (5) Encerrar o ambiente de forma limpa.

Entregável 2: Relatório Técnico (Formato PDF)

O relatório técnico deve apresentar rigor formal de publicação acadêmica, contendo:

1. **Apresentação Visual e Estatística de Dados:** Gráficos gerados programaticamente com `matplotlib` ou `seaborn`. **Requisito Obrigatório de Visualização:** Todos os gráficos de barras ou de linhas **devem incluir barras de erro** que representem o desvio padrão ou os intervalos de confiança de 95% obtidos a partir de múltiplas amostragens dos testes. Gráficos planos, sem indicação de variância estatística, serão penalizados.

2. **Consolidação Numérica:** Tabelas que sumarizam médias, medianas, desvios-padrão, p50 e p99 para TTFB, RPS e Tempo Total em cada um dos 12 testes.

3. **Análise Crítica Fundamentada na Teoria:** A discussão dos resultados não deve ser meramente descritiva. Os alunos **devem confrontar rigorosamente** os dados empíricos observados com a literatura técnica. Em especial, o relatório deve referenciar explicitamente:
    - **Tanenbaum §6.5.11 (TCP CUBIC):** Para fundamentar o colapso de throughput observado nos testes T04 e T05. Deve-se discutir analiticamente como a redução da janela de congestionamento induzida por perda e a dependência do tempo real transcorrido desde a última perda (*cubic window function*) reduzem a eficiência da banda em canais com perdas estocásticas (2% WAN).
    - **Tanenbaum §6.6.1 (QUIC) e §6.6.2 (BBR):** Para explicar por que os testes T06 e T12 mantêm alto desempenho sob as mesmas condições de degradação do canal,

detalhando como o QUIC isola a perda no fluxo UDP e utiliza abordagens de controle de congestionamento imunes à interpretação ingênua de perdas de pacotes isoladas.

A mensuração da qualidade dos entregáveis seguirá os parâmetros estabelecidos na rubrica de avaliação.

5. Rubrica Detalhada de Avaliação

A pontuação total do trabalho é de **100 pontos**, o que corresponde a 50% da nota da N2. A avaliação é estruturada em quatro eixos:

Eixo de Avaliação	Pontuação	Critérios Detalhados de Correção
1. Infraestrutura Docker e Emulação `tc netem`	**25 Pontos**	Funcionamento correto dos 3 servidores (HTTP/1.1, HTTPS/TLS 1.3, HTTP/3). Uso correto de `cap_add: NET_ADMIN`. Precisão sintática nos comandos `tc netem` garantindo RTT de 1ms (LAN) e 50ms com 2% de perda (WAN). Automação de limpeza da interface.
2. Cliente de Benchmark em Python	**25 Pontos**	Código assíncrono com `httpx` e `aioquic`. Implementação estrita do controle de vício de sessão (fechamento para cold TTFB vs canal persistente para multiplexação). Execução perfeita das Cargas A e B. Exportação obrigatória para `results.json`/`csv`.
3. Qualidade da Coleta e Visualização Gráfica	**20 Pontos**	Rigor estatístico nas medições. Cálculo preciso de TTFB, RPS, p50 e p99. Gráficos construídos em Python (`matplotlib`/`seaborn`) exibindo obrigatoriamente barras de erro (desvio padrão/intervalos de confiança), eixos rotulados e legendas claras.
4. Relatório Técnico e Discussão Teórica	**30 Pontos**	Profundidade analítica. Capacidade de explicar os gargalos empíricos com base na teoria. Citação obrigatória e articulação detalhada com **Tanenbaum §6.5.11 (TCP**

		CUBIC) e **§6.6.1 (QUIC)** para justificativa dos cenários WAN.

Tabela de Penalidades Previstas

A inobservância de requisitos técnicos resultará na aplicação direta das seguintes sanções sobre a nota final:

- **Ausência de execução automatizada por meio de um único comando (`run_experiments.sh`):** Subtração de **15 pontos**.
- **Falta de controle de vício de sessão no cliente Python (reuso indevido de conexões no teste de TTFB):** Subtração de **10 pontos**.
- **Ausência de barras de erro nos gráficos gerados:** Subtração de **10 pontos**.
- **Relatório puramente descritivo (sem fundamentação nos capítulos teóricos do Tanenbaum):** Subtração de **até 20 pontos** no Eixo 4.
- **Faltando os códigos, o relatório ou o vídeo, a nota será zero.**

As diretrizes organizacionais e as fases operacionais da entrega constam na seção final.

6. Orientações Gerais, Diretrizes Éticas e Calendário de Entrega

A condução bem-sucedida da atividade prática exige planejamento e estrita conformidade com os prazos e os padrões éticos da instituição.

Regras de Organização e Integridade Acadêmica

- **Composição das Equipes:** O trabalho poderá ser desenvolvido **individualmente ou em duplas**.
- **Política de Tolerância Zero ao Plágio:** Todos os códigos entregues e o texto do Relatório Técnico serão submetidos a sistemas automatizados de análise de similaridade. O plágio de código entre equipes, cópias não citadas de repositórios públicos ou compilações textuais sintéticas não autorizadas resultarão em **nota zero (0.0) imediata** para todos os alunos envolvidos.

Instruções de Submissão Final

A submissão final deverá ser realizada exclusivamente pelo SIGAA de acordo com a data de envio anexada à tarefa. Deve ser postado um único arquivo compactado no formato `.zip`, nomeado padronizadamente como `TP2_Redes2_NomeAluno1_NomeAluno2.zip`, contendo o diretório do código-fonte e o Relatório Técnico em PDF. O link do vídeo no Youtube deve estar no Relatório Técnico.

A engenharia de redes de alto desempenho fundamenta-se no domínio rigoroso das interações entre as camadas de protocolo. A transição do conhecimento teórico para o diagnóstico empírico é o passo definitivo para a formação de arquitetos de sistemas e engenheiros de infraestrutura preparados para atender às demandas da Internet moderna. Mãos à obra!